Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Dersin tarihi 5 Kasım 2020 Perşembe. Ders Zoom üzerinden yapıldı.
  - Asıl anlatım yaklaşık 00:11'de başlar. 01:34 ile 01:46 arasında ara var.
  - Önceki Perşembe (29 Ekim) tatil olduğu için ders yapılmadı.
  - Hoca bu konuların ileride işlenecek web madenciliği konularının ön koşulu olduğunu söyler. Veri madenciliği dersi alanlar için bu bölüm bir tekrardır.
- Sıralı örüntü madenciliği (sequential pattern mining), kitabın 2. bölümünün devamı
  - Gösterim: dizi (sequence), eleman (element), boyut (size), uzunluk (length), alt dizi (subsequence).
  - GSP algoritması ve Apriori ile benzerliği.
  - Tıklama akışı (click stream) verisinde kullanımı.
- Gözetimli öğrenme (supervised learning), Bölüm 3
  - Temel kavramlar: eğitim ve test verisi, doğruluk (accuracy), "öğrenme" tanımı.
  - Karar ağacı (decision tree).
  - Sınıf ilişkilendirme kurallarıyla sınıflandırma (classification based on class association rules).
  - Naive Bayes sınıflandırıcı (Naive Bayesian classifier).
  - k en yakın komşu (k-nearest neighbor, kNN).
- Gözetimsiz öğrenme (unsupervised learning), Bölüm 4
  - Kümeleme (clustering) ve kullanım alanları.
  - Küme kalitesi: küme içi (intra-cluster) ve kümeler arası (inter-cluster) uzaklık.
  - k-means algoritması, durma ölçütleri, karmaşıklığı, zayıf yönleri.
  - Küme gösterimi (cluster representation).
- Dönem projesi önerisi duyurusu.

Hocanın Özellikle Vurguladığı Kısımlar

- **Sıralı örüntülerin web'deki önemi:** Kullanıcının bir sitede tıklayarak izlediği gezinme örüntüsü (navigational pattern) bulunursa, kullanıcının bir sonraki adımı tahmin edilebilir.
- **Öğrenmenin ölçütü:** Bir modelin öğrendiğini söylemek için, hiç öğrenme yapılmayan durumdan daha iyi tahmin etmesi gerekir. Hoca iki sınıflı veride bunu %50 olarak verdi; doğru taban çizgisi çoğunluk sınıfının oranıdır (bkz. Ders sonrası notlar). Pratikte %80–90 beklenir.
- **Eğitim ve test verisinin dağılımı aynı olmalı.** Aksi hâlde modelin gerçek başarısı ölçülemez.
- **Karar ağacı tek değildir:** Kullanılan nitelik önem ölçütü değişince ağaç değişir. Her ağacın doğruluğu farklı olur.
- **kNN model kurmaz:** Önceki üç yöntem eğitim verisinden bir model (ağaç, kural kümesi, olasılık kümesi) çıkarır. kNN ise her yeni örnekte eğitim verisinin kendisine döner. Veri, modelin kendisidir.
- **k-means'i iyi kullanmak için:**
  - aykırı değerleri (outlier) ayıkla ya da rastgele örneklem (random sampling) kullan,
  - en uygun k'yi bul (elbow, silhouette),
  - başlangıç merkezlerini (seed) birbirinden uzak seç.
- **Kümelemede tek doğru algoritma yok:**

  - Önceden kümelerin şekli bilinmez.
  - Farklı kategorilerden algoritmalar (k-means, bisecting k-means, agglomerative) ve farklı uzaklık ölçütleri denenmeli.
  - Projede kümeleme yapan gruplardan bu karşılaştırma beklenir.
- **Proje önerisi gelecek hafta sunulacak.** Öneri şunları içermeli:
  - neden bu fikir,
  - tam olarak ne önerildiği,
  - veri kümesi,
  - değerlendirme yöntemi,
  - zaman planı (ör. 1 hafta öğrenme, 3–4 hafta geliştirme, 2–3 hafta değerlendirme ve rapor),
  - grup üyeleri (en fazla 2 kişi).

Kısa Tekrar Notları

- **Dizi:** $s = \langle a_1 a_2 \dots a_r \rangle$. Her eleman a_i bir öğe kümesidir. Elemanlar zaman sırasıyla gelir; bir eleman içindeki öğelerin sırası önemsizdir.
- **Boyut ve uzunluk:**
  - Boyut (size): dizideki eleman sayısı.
  - Uzunluk (length): dizideki toplam öğe sayısı. Uzunluğu k olan diziye k-dizi (k-sequence) denir.
  - Örnek: $\langle \{3\}\{4,5\}\{8\} \rangle$ dizisinin boyutu 3, uzunluğu 4.
- **Alt dizi:** $s_1 = \langle a_1 \dots a_r \rangle$, $s_2 = \langle b_1 \dots b_v \rangle$'nin alt dizisidir. Bunun için $1 \le j_1 < j_2 < \cdots < j_r \le v$ olacak şekilde indisler bulunmalı ve $a_1 \subseteq b_{j_1}, \ \dots, \ a_r \subseteq b_{j_r}$ olmalı. Boyutların eşit olması gerekmez.
- **Bir dizinin desteği:** O diziyi içeren müşteri dizilerinin oranı.
- **GSP:** Apriori'nin dizilere uyarlanmış hâlidir. Aday üret, veriyi tara, say, ele, döngüye dön.
- **Doğruluk:** Accuracy $= \dfrac{\text{doğru sınıflandırma sayısı}}{\text{test örneği sayısı}}$. Veri genellikle %70–80 eğitim, %20–30 test olarak ayrılır.
- **Karar ağacı:** İç düğümler nitelikler, dallar nitelik değerleri, yapraklar sınıf etiketleridir. Nitelik seçimi information gain ya da gain ratio ile yapılır.
- **Naive Bayes:**
  - $P(C \mid X) = \dfrac{P(X \mid C)\,P(C)}{P(X)}$. Karşılaştırmada $P(X)$ ortak olduğu için atılır.
  - Nitelikler sınıf verildiğinde bağımsız varsayılır: $P(X \mid C) = \prod_j P(x_j \mid C)$.
- **kNN:** Yeni örneğe en yakın k eğitim örneği bulunur. Çoğunluk sınıfı atanır.
- **k-means:**
  - k başlangıç merkezi seçilir.
  - Her nokta en yakın merkeze atanır. Merkezler yeniden hesaplanır (ortalama).
  - Durma ölçütü sağlanana kadar tekrarlanır.
  - Karmaşıklık: $O(tkn)$.
- **SSE:** SSE $= \sum_{j=1}^{k} \sum_{x \in C_j} \text{dist}(x, m_j)^2$.
- **İyi kümeleme:** küme içi uzaklık küçük, kümeler arası uzaklık büyük.

Detaylı Açıklamalar

Sıralı örüntü madenciliği

İlişkilendirme kuralı madenciliği yalnız öğelerin aynı işlemde birlikte alınıp alınmadığına bakar, alış sırasını dikkate almaz. Birçok uygulamada sıra önemlidir: - Market sepetinde, insanların hangi ürünleri hangi sırayla aldığı. - Web madenciliğinde, tıklama akışı verisi. Kullanıcı ana sayfadan başlar, bağlantılara tıklayarak sitenin diğer sayfalarına geçer. Sık gezinme örüntüleri bulunursa kullanıcının bir sonraki hareketi tahmin edilip pazarlama yapılabilir.

Alt dizi örneği (kitaptan): $\langle \{3\}\{4,5\}\{8\} \rangle$ dizisi, $\langle \{6\}\{3,7\}\{9\}\{4,5,8\}\{3,8\} \rangle$ dizisinin alt dizisidir. Çünkü $\{3\} \subseteq \{3,7\}$, $\{4,5\} \subseteq \{4,5,8\}$ ve $\{8\} \subseteq \{3,8\}$, ve bu elemanlar aynı sırayla geliyor.

İşlem verisinden müşteri dizilerine: Mağazadaki işlemler müşteriye ve zamana göre gruplanır. Her müşteri için bir veri dizisi çıkar.

Müşteri	Veri dizisi
1	$\langle\{30\}\{90\}\rangle$
2	$\langle\{10,20\}\{30\}\{40,60,70\}\rangle$
3	$\langle\{30,50,70\}\rangle$
4	$\langle\{30\}\{40,70\}\{90\}\rangle$
5	$\langle\{90\}\rangle$

minsup %25 olduğunda (5 müşteriden en az 2'si) bulunan sıralı örüntüler: - 1-diziler: $\langle\{30\}\rangle, \langle\{40\}\rangle, \langle\{70\}\rangle, \langle\{90\}\rangle$ - 2-diziler: $\langle\{30\}\{40\}\rangle, \langle\{30\}\{70\}\rangle, \langle\{30\}\{90\}\rangle, \langle\{40,70\}\rangle$ - 3-dizi: $\langle\{30\}\{40,70\}\rangle$. Bu dizi 2. ve 4. müşterinin dizilerinde geçer, destek $2/5 = \%40$.

Bu örüntüden "30 alındıktan sonra 40 ve 70 birlikte alınır" diye bir sıralı kural çıkarılabilir. Müşteri 2'de $\{40,60,70\}$ elemanının 30'dan sonra alındığı bilinir. Ancak 40, 60 ve 70'in kendi aralarındaki sırası bilinmez.

GSP algoritması: Apriori ile aynı iskelete sahiptir. Sıralı örüntü algoritmalarının çoğu Apriori'den türetilmiştir. 1. İlk taramada 1-diziler sayılır, sık olanlar bulunur. 2. Döngüde aday k-diziler üretilir, veri yeniden taranıp sayılır, sık olanlar seçilir. 3. En büyük sık diziye ulaşılınca durulur. Bu diziden sıralı kurallar çıkarılır.

Tıklama akışı örneği: Her oturum (session) bir işlem gibi düşünülür. Örnekte kullanıcı t=0'da A sayfasına gider, 3t'de A'yı yeniler, 4t'de B'ye geçer. Tüm oturumlara sıralı örüntü madenciliği uygulanınca çok sayıda kullanıcının izlediği farklı uzunluktaki sık gezinme dizileri bulunur.

Gözetimli öğrenme: temel kavramlar

Gözetimli öğrenmede etiketli (labeled) veriden öğrenilir ve yeni bir verinin sınıfı (class / label) tahmin edilir. "Gözetim" etiketli veriden gelir.

Hocanın örnekleri: - **Acil servis:** Yoğun bakım yatağı sınırlıdır. Hastanın yüksek riskli mi düşük riskli mi olduğuna karar veren bir model gerekir. - **Kredi kartı başvurusu:** Yaş, medeni durum, yıllık gelir gibi bilgilerden "onayla / onaylama" kararı verilir.

Süreç: 1. Eğitim verisine (training data) bir öğrenme algoritması uygulanır ve bir model elde edilir. 2. Model, etiketleri bilinen test verisinde, etiketler bilinmiyormuş gibi çalıştırılır. 3. Tahmin edilen etiketler gerçek etiketlerle karşılaştırılır ve doğruluk hesaplanır.

"Öğrenme" tanımı: - Bileşenler: veri kümesi D, görev T, başarım ölçüsü M. - Bir sistem, D'den öğrendikten sonra T görevindeki M ile ölçülen başarımı artıyorsa öğrenmiş sayılır. - Kredi örneğinde D başvuru verisi, T onay tahmini, M doğruluktur. - Kitaptaki 15 örneklik kredi verisinde 9 örnek "yes", 6 örnek "no". - Hiç öğrenmeden her yeni örneğe çoğunluk sınıfı "yes" denirse doğruluk $9/15 = \%60$ olur. - Bu yüzden bir öğrenme algoritması %60'tan daha iyi sonuç vermelidir.

Hoca bazı algoritmaların ayrıntısının veri madenciliği ve makine öğrenmesi derslerinde öğrenileceğini söyler. Burada amaç, öğrencilerin dönem projesinde bu algoritmaları web verisine uygulayabilmesidir.

Karar ağacı

Model bir ağaç veri yapısıdır. Kredi verisi için örnek ağaç: - Kökte Age bulunur. - Age = young dalında Has_job, middle dalında Own_house, old dalında Credit_rating bulunur. - Yapraklarda Yes/No etiketleri yer alır.

Kurma: - Başta tüm veri kök düğümdedir. - Bir nitelik önem ölçütüyle (impurity function), örneğin information gain ya da gain ratio ile her niteliğin veriyi dallara ne kadar iyi ayırdığı ölçülür. - En iyi nitelik düğüme konur. Veri, bu niteliğin değerlerine göre dallara dağıtılır. Örneğin "young" olan 5 örnek ilgili dala gider. - Her yaprakta çoğunluk sınıfı etiket olur.

Algoritma böl ve yönet (divide and conquer) yaklaşımıyla çalışır. Durma koşulları: - Düğümdeki tüm örnekler aynı sınıftandır. - Kullanılacak nitelik kalmamıştır. - Örnek kalmamıştır.

Tahmin: Yeni örnek kökten başlayıp değerlerine göre dallardan inerek bir yaprağa ulaşır. Yaprağın etiketi tahmin olur. Örneğin Age = young ve Has_job = false olan örnek "No" yaprağına düşer.

Nitelik seçiminin sezgisi (yanlış yerleştirilen örnek sayısı): - Age seçilirse: - Dallar young, middle, old. Her dalda 5 örnek var. - Çoğunluk etiketleri sırasıyla No, Yes, Yes. - Yanlış yerleştirilen örnek sayısı $2 + 2 + 1 = 5$. - Own_house seçilirse: - Dallar true ve false. - true dalında 6 örneğin hepsi Yes, hata 0. - false dalında 9 örnek var, çoğunluğu No, 3 örnek Yes. - Toplam hata 3. - Own_house daha az hata verdiği için bu düğüm için daha iyi bir seçimdir.

Ağaçtan kurallar da çıkarılabilir. Kökten başlayarak sıralanmış bu kural zinciri model olarak kullanılabilir. Hoca C4.5'in sözde kodunu gösterir.

Sınıf ilişkilendirme kurallarıyla sınıflandırma

- Önceki bölümdeki sınıf ilişkilendirme kuralı madenciliğiyle, verilen minsup ve minconf için sağ tarafı sınıf etiketi olan kurallar çıkarılır.
- Kurallar confidence ve support değerlerine göre büyükten küçüğe sıralanır.
- Yeni örnek için sırayla kurallara bakılır. Örneği ilk kapsayan kuralın sınıfı atanır.
- Hiçbir kural örneği kapsamıyorsa varsayılan sınıf (default class) atanır.
- Burada model bir kural kümesidir.

Naive Bayes sınıflandırıcı

Bu yöntemde model, veriden hesaplanan olasılıklar kümesidir. Bayes teoremine göre:

$$P(C_i \mid X) = \frac{P(X \mid C_i)\, P(C_i)}{P(X)}$$

Her sınıf için bu değer hesaplanıp en büyüğü seçilir. Payda $P(X)$ tüm sınıflarda aynı olduğu için karşılaştırmada atılır. Yalnız $P(X \mid C_i)P(C_i)$ hesaplanır.

Örnek (buys_computer, 14 örnek): $X = (\text{age} \leq 30,\ \text{income} = \text{medium},\ \text{student} = \text{yes},\ \text{credit_rating} = \text{fair})$

- $P(\text{yes}) = 9/14 = 0{,}643$, $P(\text{no}) = 5/14 = 0{,}357$.
- yes sınıfı için koşullu olasılıklar:
  - $P(\text{age} \leq 30 \mid \text{yes}) = 2/9 = 0{,}222$
  - $P(\text{medium} \mid \text{yes}) = 4/9 = 0{,}444$
  - $P(\text{student} \mid \text{yes}) = 6/9 = 0{,}667$
  - $P(\text{fair} \mid \text{yes}) = 6/9 = 0{,}667$
- no sınıfı için koşullu olasılıklar:
  - $P(\text{age} \leq 30 \mid \text{no}) = 3/5 = 0{,}6$
  - $P(\text{medium} \mid \text{no}) = 2/5 = 0{,}4$
  - $P(\text{student} \mid \text{no}) = 1/5 = 0{,}2$
  - $P(\text{fair} \mid \text{no}) = 2/5 = 0{,}4$
- $P(X \mid \text{yes}) = 0{,}222 \times 0{,}444 \times 0{,}667 \times 0{,}667 \approx 0{,}044$
- $P(X \mid \text{no}) = 0{,}6 \times 0{,}4 \times 0{,}2 \times 0{,}4 = 0{,}019$
- $P(X \mid \text{yes})P(\text{yes}) \approx 0{,}028$
- $P(X \mid \text{no})P(\text{no}) \approx 0{,}007$
- Sonuç: X, "buys_computer = yes" sınıfına atanır.

Değerler Python ile hesaplanıp doğrulandı.

Kolay öğrenilen, kolay uygulanan bir algoritmadır. Gereken tüm olasılıklar doğrudan veriden sayılır.

k en yakın komşu (kNN)

- Kullanıcı bir k verir, örneğin $k = 6$.
- Yeni örnek ile eğitim verisindeki her örnek arasındaki uzaklık bir uzaklık fonksiyonuyla hesaplanır.
- En yakın k örnek bulunur. Bunların çoğunluk sınıfı yeni örneğe atanır.
- Model kurulmaz. Eğitim ve tahmin iç içedir. Her tahminde eğitim verisine dönülür.

Gözetimsiz öğrenme: kümeleme

Veride etiket ya da hedef nitelik (target attribute) yoktur. Amaç birlikte gruplanabilecek veri öğelerini, yani kümeleri, keşfetmektir. - İki boyutlu verilerde kümeler gözle görülebilir. Gerçek veride boyut sayısı çok yüksektir. Bu yüzden algoritmalara ihtiyaç vardır. - **Kullanım alanları:** - Üretim: tişört bedenlerini müşteri gruplarına göre belirlemek. - Pazarlama: müşteri segmentlerini bulmak. - Belgeleri içeriklerine göre kategorilere ayırmak. - Web madenciliği: sitedeki kullanıcı davranışını ve sıcak noktaları (en çok kullanılan sayfaları ve işlevleri) anlamak. - **Kategoriler:** Bölümlemeli (partitional) ve hiyerarşik (hierarchical) kümeleme. Hoca ayrıca dağılıma dayalı (distribution-based) kümelemeden söz eder. - **Kalite:** Küme içi uzaklık (kümedeki öğeler arasındaki ortalama uzaklık) küçük olmalı. Kümeler arası uzaklık büyük olmalı. Sonucun kalitesi algoritmaya, veriye ve uzaklık fonksiyonuna bağlıdır.

k-means

Veri kümesi $D = \{x_1, \dots, x_n\}$, her x_i r boyutlu bir vektördür. Kullanıcı küme sayısı k'yi baştan verir.

```
Algorithm k-means(k, D)
  choose k data points as the initial centroids (cluster centers)
  repeat
      for each data point x in D do
          compute the distance from x to each centroid
          assign x to the closest centroid
      re-compute the centroids using the current cluster memberships
  until the stopping criterion is met
```

Örnekte $k = 2$. Başta iki gerçek veri noktası merkez seçilir. Atama ve merkez hesaplama tekrarlandıkça merkezler gerçek veri noktası olmayan "sanal" noktalara kayar. Üçüncü turda merkezler artık değişmez.

Durma ölçütleri: 1. Noktaların kümeler arasında yeniden atanması yok ya da çok az. 2. Merkezlerde değişim yok ya da çok az. 3. SSE'deki azalma çok küçük.

Uzaklık: En yaygın olanı Öklid uzaklığıdır (Euclidean distance). Niteliklerin sayısal olduğunu varsayar.

Disk üzerindeki veride k-means: - Verinin tamamı belleğe alınmaz. Her turda veri bir kez taranır. - Her küme için bir toplam (sum) ve bir sayaç (count) tutulur. - Her nokta en yakın merkeze atanınca o kümenin toplamı ve sayacı güncellenir. - Tur sonunda yeni merkez $\text{sum}_j/\text{count}_j$ olur.

Karmaşıklık: $O(tkn)$. Burada t tur sayısı, k küme sayısı, n veri sayısıdır. t ve k küçük olduğu için algoritma pratikte doğrusaldır.

Zayıf yönler ve çözümler: - k **önceden bilinmeli.** Çözüm: elbow ya da silhouette yöntemleriyle $k = 2, 3, 4, \dots$ denenip en iyi kümelemeyi veren k seçilir. - **Aykırı değerlere duyarlı.** Tek bir aykırı nokta kümeleri bozabilir. Çözümler: - Önce aykırı değerleri bulup veriden çıkarmak. - Ya da rastgele örneklem almak. Hocanın örneği: 1 milyon noktadan %10'u belleğe alınır ve kümelenir, diyelim 5 küme bulunur. Kalan noktalar yeniden kümelenmeden bu 5 kümenin en yakın merkezine atanır. Örneklemde aykırı değer çıkma olasılığı düşüktür. - **Yalnız ortalaması tanımlı (sayısal) veriye uygulanır.** - **Başlangıç merkezlerine duyarlı.** Kötü seçilen tohumlar doğal olmayan kümeler verir. Hocanın yöntemi: - İlk tohum seçilir. - İkinci tohum, ilkinden en uzak nokta olur. - Üçüncü tohum, ilk ikisine ortalamada en uzak nokta olur. - Böyle

devam edilir. - **Yalnız yuvarlak ya da eliptik kümeleri bulabilir.** Farklı şekilli kümeler için hiyerarşik kümeleme gibi başka yöntemler gerekir. Bu konu bir sonraki derste işlenecek.

Küme gösterimi: Kümelerin tüm noktaları bellekte tutulmaz. Küme şu yollarla özetlenir: - Merkez ve yarıçap. - Her boyut için sınır değerleri. Örnek: "1. küme: $x \leq 2$", "2. küme: $x > 2$ ve $y > 1{,}5$". - En sık geçen öğe. Metin kümelemede kümeyi en sık geçen sözcükler temsil eder.

Dersin devamı ve proje fikirleri

- Sonraki derste hiyerarşik kümeleme bitirilip Bölüm 6'ya (Information Retrieval and Web Search, web içerik madenciliği) geçilecek.
- Ardından bağlantı analizi (web yapı madenciliği), web taraması ve web kullanım madenciliği gelecek. Vakit kalırsa görüş madenciliği ve duygu analizi (opinion mining, sentiment analysis) de işlenecek.
- **Proje fikirleri:**
  - Belirli bir tarama algoritmasıyla web'den veri indiren bir crawler.
  - Web'den çekilen belgeler üzerinde kümeleme.
  - Kullanıcı verisinden öğrenen gözetimli bir uygulama.
  - Hibrit arama: farklı e-ticaret sitelerinden aynı ürün grubunu sürekli tarayıp fiyatlarıyla karşılaştırmalı listeleyen bir sayfa.
  - Hazır web veri kümeleri üzerinde bölümlerdeki algoritmaları çalıştırmak.
- **Önceki yıllardan örnek bir proje:**
  - Dinamik URL'ler toplanıp elle etiketlendi.
  - URL'ler parçalara ayrıldı. Parçalar word2vec ile vektörlere dönüştürüldü.
  - Vektörlere bölümlemeli ve hiyerarşik kümeleme, farklı uzaklık ölçütleriyle uygulandı.
  - Sonuçlar görselleştirilip başarımları ölçüldü. Proje bir makaleye dönüştü.

Ders sonrası notlar

- **Alt dizilerde boyut eşitliği:** Bir öğrenci "s_1'in s_2'nin alt dizisi olması için boyutları eşit mi olmalı?" diye sordu. Hoca "tabii" diye yanıtladı. Bu yanlış.
  - Tanımda $r \leq v$'dir; s_2 daha fazla eleman içerebilir.
  - Yalnız s_1'in her elemanı, s_2'nin sırayı koruyan farklı bir elemanının alt kümesi olmalıdır.
  - Kitaptaki örnekte boyutu 3 olan $\langle\{3\}\{4,5\}\{8\}\rangle$, boyutu 5 olan bir dizinin alt dizisidir.
- **Karar ağacında nitelik sırası:** Hoca nitelikleri bir kez information gain'e göre sıralayıp en önemlisini köke, sonrakileri alt düğümlere koyduğumuzu anlatır. Algoritma böyle çalışmaz.
  - Information gain her düğümde, o düğüme düşen veri alt kümesi için yeniden hesaplanır.
  - O alt küme için en iyi nitelik seçilir.
  - Bu yüzden farklı dallarda farklı nitelikler çıkar. Örnekte young dalında Has_job, old dalında Credit_rating var.
- **Böl ve yönet:** Hoca böl ve yönetin veriyi her adımda ikiye böldüğünü söyler. Karar ağacında veri, seçilen niteliğin değer sayısı kadar parçaya bölünür. Age için bu üç parçadır.
- **Kümeler arası ve küme içi uzaklık:** 01:54 civarında hoca bir an tersini söyler ("inter-cluster minimize, intra-cluster maximize"). Doğrusu: küme içi uzaklık en küçük, kümeler arası uzaklık en büyük olmalıdır. Hoca bunu hemen öncesinde doğru anlatmıştır.
- **Hiyerarşik ve dağılıma dayalı kümeleme:** Bunlar ayrı kategorilerdir. Hiyerarşik kümeleme bir ağaç (dendrogram) üretir. Dağılıma dayalı kümeleme (ör. Gauss karışım modelleri) veriyi olasılık dağılımlarıyla açıklar.
- **Elbow ve silhouette:** Bunlar ayrı "algoritmalar" değil, k seçme yöntemleridir.
  - Elbow yönteminde k artarken SSE'nin azalmasının belirgin biçimde yavaşladığı "dirsek" noktası seçilir. "En yüksek kaliteli k" seçilmez; SSE k arttıkça hep azalır.
  - Silhouette yönteminde ortalama silhouette değeri en yüksek olan k seçilir.
- **Tohum seçimi:** Hocanın anlattığı "en uzak nokta" yöntemi (farthest-first) aykırı değerlere duyarlıdır. En uzak nokta çoğu zaman bir aykırı değer olur. Bugün yaygın kullanılan k-means++ yönteminde tohumlar, uzaklığın karesiyle orantılı olasılıkla rastgele seçilir.
- **Kredi verisinde "öğrenmesiz" doğruluk:** Hoca önce "iki sınıf varsa %50" der, sonra çoğunluk sınıfına göre %60 hesaplar. Doğru taban çizgisi (baseline) çoğunluk sınıfının

oranıdır: $9/15 = \%60$.

- **Altyazı hataları:**
  - 00:58 civarındaki “bu kategori için” tekrarları ve 01:19–01:20 arasındaki “the biggest thing is that” tekrarları anlamsızdır.
  - “Naive-Payacin”, “Biosyn” Naive Bayesian / Bayes; “bias computer” buys_computer; “divide and contour” divide and conquer; “ELBOV” elbow; “Suleid” silhouette; “K-Mins” k-means demektir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.